เอกสารส่งมอบงาน

ฉบับที่ 1 — สภาพแวดล้อมการทำงานที่ทำซ้ำได้ (Reproducible Runtime)

หัวข้อ	รายละเอียด
วันที่จัดทำ	8 พฤศจิกายน 2569
ผู้ส่งมอบ	..... (ระบุชื่อ-นามสกุล)
ผู้รับมอบ	..... (ระบุชื่อ-นามสกุล)
ขอบเขตงาน	การทำให้ระบบ GIS ประกอบขึ้นใหม่และทำงานได้บนเครื่องอื่น
บรานช์ที่เกี่ยวข้อง	gis-unlock (อยู่บน GitHub แล้ว) ต่อยอดจาก feature/gis-data-management
สถานะ	ยังไม่รวมเข้ากับบรานช์หลัก (main)

1. บทสรุปสำหรับผู้รับมอบ

ระบบจัดการข้อมูล GIS ได้รับการพัฒนาจนทำงานได้ครบวงจรบนเครื่องของผู้พัฒนา และมีชุดทดสอบอัตโนมัติจำนวน 2,089 รายการที่ผ่านการตรวจสอบทั้งหมดบนระบบตรวจอัตโนมัติของ GitHub

อย่างไรก็ตาม ส่วนประกอบที่จำเป็นต่อการรันระบบบางรายการไม่ได้ถูกบันทึกไว้ในคลังโค้ด ทำให้บุคคลอื่นไม่สามารถประกอบระบบขึ้นมาใหม่ได้ งานที่ส่งมอบตามเอกสารฉบับนี้คือการปิดช่องว่างดังกล่าว

งานทั้งหมดเป็นงานเขียนโค้ดและตั้งค่า ซึ่งไม่ขึ้นกับการตัดสินใจของผู้ใด สามารถเริ่มดำเนินการได้ทันที และมีเกณฑ์ตัดสินความสำเร็จที่ชัดเจนจากชุดทดสอบที่มีอยู่แล้ว

2. ปัญหาที่ตรวจพบ พร้อมหลักฐาน

2.1 ไม่มีผู้ใดสามารถรันกระบวนการประมวลผล (worker) ได้ นอกจากเครื่องของผู้พัฒนาเดิม

ระบบเรียกใช้ container image ด้วยค่า digest โดยตรง โดยไม่มีชื่อกำกับและไม่มีไฟล์ Dockerfile ประกอบ

```
dev/gis_local/manage.py:47   WORKER_IMAGE = "sha256:fca4560d8a0d...aa04c9"
dev/gis_local/manage.py:48   GEE_MOCK_WORKER_IMAGE = "sha256:7557955c9e2c...9566674"
```

ผลการตรวจสอบบนเครื่องผู้จัดทำเอกสาร

```
$ find . -iname "Dockerfile*" -not -path "*/node_modules/*"
./green-area-backend/Dockerfile
./green-area-frontend/Dockerfile
```

```
$ docker image inspect sha256:fca4560d8a0d...aa04c9
NOT ON THIS MACHINE
$ docker image inspect sha256:7557955c9e2c...9566674
NOT ON THIS MACHINE
```

ไฟล์ Dockerfile ที่พบเป็นของเว็บเซอร์วิสและของส่วนหน้าเว็บเท่านั้น ไม่พบของ worker และ image ทั้งสองรายการไม่มีอยู่บนเครื่องที่ตรวจสอบ

เอกสาร docs/gis/a6-3-handoff.md บันทึกไว้เพียงว่าใช้ digest ใด แต่ไม่ได้ระบุวิธีสร้าง

ผลกระทบ หาก image ดังกล่าวสูญหาย จะไม่มีวิธีสร้างขึ้นใหม่ให้เหมือนเดิม

2.2 สภาพแวดล้อมของ sandbox ก็ประกอบขึ้นใหม่ไม่ได้เช่นกัน

ไฟล์ dev/gis_local/compose.yaml กำหนด image ผ่านตัวแปรสภาพแวดล้อมแบบบังคับ

```
image: ${GIS_LOCAL_DB_IMAGE:?pinned image required}
image: ${GIS_LOCAL_AUTH_IMAGE:?pinned image required}
image: ${GIS_LOCAL_REST_IMAGE:?pinned image required}
image: ${GIS_LOCAL_GATEWAY_IMAGE:?pinned image required}
```

ค้นทั้งคลังโค้ดแล้วไม่พบค่าจริงของตัวแปรทั้งสี่รายการ จึงไม่สามารถสั่ง docker compose up ได้

2.3 ค่า hash ที่ตรึงไว้ขึ้นกับไลบรารีของระบบ ไม่ใช่เพียงรุ่นของแพ็กเกจ

ชุดทดสอบ test_v1_artifacts_of_existing_fixtures_are_byte_identical ตรวจว่าผลลัพธ์จากการแปลงไฟล์ shapefile ต้องมีค่า hash ตรงกับที่ตรึงไว้ทุกไบต์ ผลการทดลองเปรียบเทียบสามสภาพแวดล้อม

สภาพแวดล้อม	รุ่นไลบรารี	ผลการทดสอบ
python:3.12-slim (Docker)	fiona 1.10.1 / GDAL 3.9.2, shapely 2.2.0 / GEOS 3.14.1	ผ่าน
GitHub ubuntu-24.04	รุ่นเดียวกันทุกรายการ (ไฟล์ wheel เดียวกัน)	ไม่ผ่าน
Windows 11	รุ่นเดียวกัน แต่ wheel คนละ build	ไม่ผ่าน

ตัวอย่างค่าที่ต่างกันบน GitHub runner

```
expected  c8201dded27c987f7d2979cf78b33fcb987289bf8acbbed56ad836d43a623085
actual    12ff267b84767b20daef64698262078659ee3e8848bc359953bc5cc485b90a41
```

ข้อสรุป การตรึงรุ่นไลบรารีในไฟล์ requirements-native.txt เป็นสิ่งจำเป็นแต่ยังไม่เพียงพอ ต้องตรึงทั้ง image จึงจะรับประกันผลลัพธ์ได้ ปัจจุบันชุดทดสอบนี้ถูกตั้งให้ข้ามโดยอัตโนมัติ และจะทำงานก็ต่อเมื่อประกาศตัวแปร GIS_NATIVE_RUNTIME=pinned เท่านั้น

2.4 ไฟล์ที่ผู้ใช้อัปโหลดถูกเก็บบนดิสก์ของเครื่อง

ไฟล์ gis_ldd_storage.py และ gis_geotiff_storage.py เขียนไฟล์ลงดิสก์พร้อมไฟล์ owner.json กำกับความเป็นเจ้าของ และตรวจสอบค่า hash ทุกครั้งที่อ่าน (รหัสข้อผิดพลาด GIS_LDD_INTEGRITY) เมื่อนำขึ้นโฮสต์ประเภท Render ดิสก์จะถูกล้างทุกครั้งที่เริ่มระบบใหม่

3. รายการงานที่ส่งมอบ

หมายเหตุ วันกำหนดส่งเป็นข้อเสนอเบื้องต้น ขอให้ยืนยันร่วมกับหัวหน้าโครงงานอีกครั้ง

รหัส	งาน	สิ่งที่ต้องส่ง	เกณฑ์ว่าเสร็จ	กำหนดส่ง
B1	บันทึก image ที่ใช้อยู่ลงคลังโค้ด	ไฟล์ dev/gis_local/images.json ระบุ digest ทั้ง 6 รายการ (sandbox 4 + worker 2)	สั่ง docker compose up ได้โดยไม่ต้องสอบถามผู้ใด	12 พ.ย. 2569
B2	สร้าง Dockerfile ของ worker	green-area-backend/gis_worker/Dockerfile พร้อมคำสั่ง build	รันชุดทดสอบใน image นั้นโดยตั้ง GIS_NATIVE_RUNTIME=pinned แล้วผ่านทุกข้อ	20 พ.ย. 2569
B3	ตัวรัน migration สำหรับฐานข้อมูลบนคลาวด์	สคริปต์ติดตั้ง migration 027–048 รวดเดียว พร้อมบันทึก ledger	ติดตั้งบนฐานข้อมูลเปล่าได้สำเร็จ และ gis_readiness() รายงานตัวตนถูกต้อง	27 พ.ย. 2569
B4	ย้ายที่จัดเก็บไฟล์อัปโหลด	แก้ไข gis_ldd_storage.py และ gis_geotiff_storage.py ให้เก็บบนบริการภายนอก	ชุดทดสอบเดิมผ่านทั้งหมด และการตรวจสอบ hash ยังทำงาน	11 ธ.ค. 2569

4. รายละเอียดประกอบรายการงาน

งาน B1

ขอค่า digest จากผู้พัฒนาเดิม แล้วบันทึกลงคลังโค้ด เป็นงานที่ใช้เวลาไม่นาน แต่ควรดำเนินการก่อนงานอื่นทั้งหมด เนื่องจากปัจจุบันข้อมูลนี้อยู่ในความครอบครองของบุคคลเพียงผู้เดียว

งาน B2

สูตรต่อไปนี้ผ่านการทดสอบแล้ว และสามารถใช้เป็นจุดตั้งต้นได้

```
FROM python:3.11-slim
```

```
RUN apt-get update && apt-get install -y --no-install-recommends libexpat1 \
    && rm -rf /var/lib/apt/lists/*
COPY requirements-native.txt .
RUN pip install --no-cache-dir -r requirements-native.txt
ENV GIS_NATIVE_RUNTIME=pinned
```

ยืนยันแล้วว่าได้ Rasterio 1.4.4 / GDAL 3.10.3, Fiona 1.10.1 / GDAL 3.9.2, Shapely 2.2.0 / GEOS 3.14.1 และ NumPy 2.4.6 ซึ่งตรงกับที่บันทึกไว้ใน docs/gis/t3-remediation-handoff.md ทุกรายการ

ข้อควรระวัง การสร้าง image สำเร็จไม่ได้หมายความว่าถูกต้อง ต้องผ่านชุดทดสอบตามข้อ 2.3 ด้วย หากไม่ผ่าน แสดงว่า image ยังไม่ตรงกับสภาพแวดล้อมที่ค่า hash ถูกตรึงไว้

งาน B3

ปัจจุบันการติดตั้ง migration กระจายอยู่ในไฟล์ dev/gis_local/*_local.py ประมาณ 8 ไฟล์ โดยแต่ละไฟล์ฝังหมายเลขเวอร์ชันของตนเอง ยังไม่มีตัวรันเดียวที่ครอบคลุม 027 ถึง 048

ตัวรันต้องประกาศค่าสองรายการก่อนรันเนื้อ migration

```
SELECT set_config('gis.cluster_name', '<project-ref>', true);
SELECT set_config('gis.mode', 'cloud', true);
```

โดย <project-ref> คือรหัสโครงการของ Supabase ซึ่งเป็นตัวอักษรพิมพ์เล็ก 20 ตัว หากไม่ประกาศค่าทั้งสอง ระบบจะถือว่าเป็น sandbox ในเครื่อง ซึ่งเป็นพฤติกรรมเดิมที่ยังคงไว้

สิ่งที่ต้องตรวจสอบก่อนเริ่ม (ควรเสร็จภายใน 11 พ.ย. 2569 เนื่องจากกระทบงานอื่นทั้งหมด)

- ฐานข้อมูล Supabase ของโครงงานเป็น PostgreSQL รุ่น 17 ขึ้นไปหรือไม่ เนื่องจาก migration 027 กำหนดเงื่อนไข server_version_num >= 170000 หากไม่ถึงจะติดตั้งไม่ได้
- สิทธิ์ในการสร้าง role เนื่องจาก migration 027 และ 043 ถึง 047 มีคำสั่ง CREATE ROLE สำหรับ gis_owner, gis_worker, gis_native_worker และ authenticator ต้องตรวจสอบว่าบัญชีที่ใช้มีสิทธิ์เพียงพอ หากไม่มี ต้องแยกออกเป็นขั้นตอนตั้งค่าต่างหาก

งาน B4

การย้ายที่จัดเก็บต้องคงกลไกตรวจสอบความถูกต้องของไฟล์เดิมไว้ทั้งหมด ไม่อนุญาตให้ตัดการตรวจสอบค่า hash ออกเพื่อความสะดวก

5. สิ่งที่จัดเตรียมไว้ให้แล้ว

รายการ	สถานะ
requirements-native.txt ระบุรุ่นไลบรารีที่ตรวจสอบแล้ว	เพิ่มเข้าคลังโค้ดแล้ว

รายการ	สถานะ
ขั้นตอนติดตั้งไลบรารีในระบบตรวจอัตโนมัติ	เพิ่มใน .github/workflows/ci.yml แล้ว
ช่องทางประกาศเป้าหมาย (GIS_TARGET, GIS_API_URL และอื่น ๆ)	ใช้งานได้แล้ว ดู green-area-backend/.env.example
ตัวอย่างการตั้งค่าฝั่งเว็บ	ดู green-area-frontend/.env.example
ระบบตรวจอัตโนมัติ (CI)	แก้ไขจนผ่านทุกขั้นตอนแล้ว

6. วิธีตรวจสอบสภาพปัจจุบันด้วยตนเอง

```
git clone <repo> && cd <repo>
git checkout gis-unlock


cd green-area-backend
pip install -r requirements.txt
pip install -r requirements-native.txt
ruff check .
pytest tests/ -q


cd ../green-area-frontend
npm ci
npx eslint src --max-warnings 0
npm test
npm run build
```

เกณฑ์ที่ถือว่าผ่าน

- การติดตั้ง requirements-native.txt ต้องมี libexpat1 อยู่ในระบบก่อน
- คำสั่ง ruff check . ต้องแสดงข้อความ All checks passed
- คำสั่ง pytest ต้องไม่มีรายการที่ไม่ผ่าน
- คำสั่ง eslint ต้องไม่มีข้อผิดพลาดและไม่มีคำเตือน

7. แบบบันทึกความคืบหน้า

รหัส	ผู้รับผิดชอบ	วันที่เริ่ม	วันที่เสร็จ	ผลการตรวจ
B1				ผ่าน / ไม่ผ่าน
B2				ผ่าน / ไม่ผ่าน
B3				ผ่าน / ไม่ผ่าน
B4				ผ่าน / ไม่ผ่าน

ผู้ส่งมอบ ..... วันที่ ..... / ..... / .....

ผู้รับมอบ ..... วันที่ ..... / ..... / .....

ผู้รับทราบ (หัวหน้าโครงงาน) ..... วันที่ ..... / ..... / .....